

SPI Slave with Single Port RAM

Prepared by: Aly Khaled

Table of Contents:

Abstract.....	3
Introduction.....	4
Project Overview.....	4
System Flow.....	4
System Architecture.....	5
State Diagram.....	5
RTL Code.....	6
RAM.....	6
SPI Slave.....	10
SPI Wrapper.....	11
Test Bench.....	13
DO File.....	14
Questa Sim Waveform.....	14
Linting.....	14
Constraint File.....	15
Elaborated Design.....	16
FSM Encoding.....	17
One Hot.....	17
Gray.....	19
Sequential.....	21

Abstract:

This project presents the design and implementation of a Serial Peripheral Interface (SPI) slave integrated with a single-port synchronous RAM using Verilog HDL. The SPI slave communicates with an external master through standard SPI signals and transfers data to or from the RAM under the control of a finite state machine (FSM). The system is wrapped in a top-level module that connects the RAM and the SPI slave, enabling both write and read operations. Simulation and verification were performed using Questa Sim, where waveform analysis confirmed the correct operation of address loading, data storage, and data retrieval. The project demonstrates how digital design concepts such as synchronous memory, FSM control, and serial communication protocols can be combined to build a functional hardware module.

Introduction:

Project Overview:

The project implements a digital system that allows communication between an SPI master and a single-port synchronous RAM through an SPI slave interface. The design is written in Verilog HDL and organized into three main modules:

- **Single-Port Synchronous RAM:** Stores 8-bit data at 256 memory locations, controlled by read and write commands.
- **SPI Slave:** Receives serial data (MOSI), decodes commands with a finite state machine (FSM), and sends or receives data through MISO.
- **Wrapper Module:** Integrates the RAM and SPI slave to form a complete working system.

The system supports four main operations:

1. Load a write address.
2. Write data to RAM.
3. Load a read address.
4. Read data from RAM.

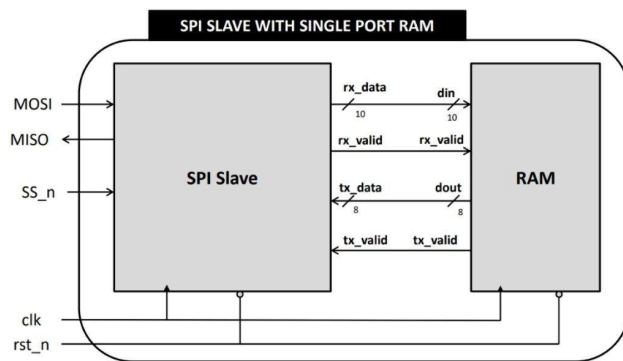
System Flow:

When the SPI master asserts the Slave Select (SS_n) line, the slave begins communication. Incoming serial bits are shifted into the SPI slave and interpreted as either an address or data, depending on the command. These inputs are then used to access the RAM, and in the case of a read operation, the stored data is returned to the master through the MISO line.

System Architecture:

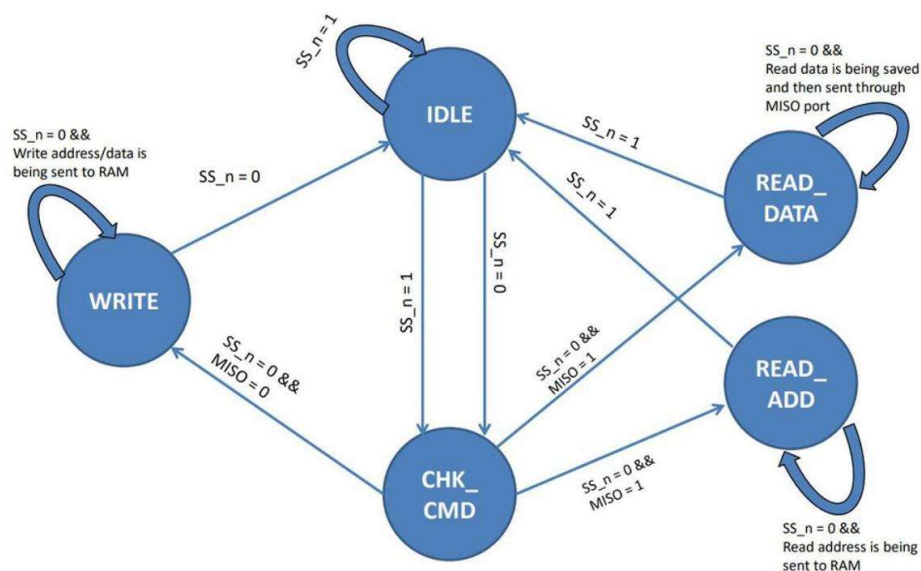
The wrapper module coordinates the SPI slave and RAM to perform read and write operations. The key signals that control data flow are:

- **rx_data**: 10-bit input from the SPI slave representing incoming commands or data.
- **rx_valid**: Indicates that rx_data is ready to be processed by the RAM.
- **tx_data**: 8-bit data read from RAM to be sent back to the SPI master.
- **tx_valid**: Signals that tx_data is ready for transmission.



State Diagram:

The diagram represents the operation flow of the SPI slave as it responds to SPI commands and manages RAM transactions.



RTL Code:

RAM:

```
1  module single_port_sync_ram (din,rx_valid,clk,rst_n,tx_valid,dout);
2  parameter MEM_DEPTH=256;
3  parameter ADDR_SIZE=8;
4  input [9:0] din;
5  input rx_valid;
6  input clk;
7  input rst_n;           //synchronous active-low reset.
8  output reg tx_valid;
9  output reg [7:0] dout;
10 reg [ADDR_SIZE-1:0] addr_wr;
11 reg [ADDR_SIZE-1:0] addr_rd;
12 reg [7:0] mem [MEM_DEPTH-1:0];
13 always @ (posedge clk) begin
14   if(~rst_n) begin
15     dout<=0;
16     tx_valid<=0;
17     addr_rd<=0;
18     addr_wr<=0;
19   end
20   else begin
21     if(rx_valid) begin
22       case(din[9:8])
23       2'b00: begin
24         addr_wr<=din[7:0];
25         tx_valid<=0;
26       end
27       2'b01: begin
28         mem[addr_wr]<=din[7:0];
29         tx_valid<=0;
30       end
31       2'b10: begin
32         addr_rd<=din[7:0];
33         tx_valid<=0;
34       end
35       2'b11: begin
36         dout<=mem[addr_rd];
37         tx_valid<=1;
38       end
39     endcase
40   end
41 end
42 end
43 endmodule
44
```

SPI Slave:

```
45 module SPI_slave (MOSI,SS_n,tx_data,tx_valid,clk,rst_n,MISO,rx_data,rx_valid);
46 //parameters
47 parameter IDLE=3'b000;
48 parameter CHECK_COMMAND=3'b001;
49 parameter WRITE=3'b010;
50 parameter READ_ADD=3'b011;
51 parameter READ_DATA=3'b100;
52 //inputs
53 input MOSI;
54 input SS_n;
55 input clk;
56 input rst_n; //synchronous active-low reset.
57 input [7:0] tx_data;
58 input tx_valid;
59 //outputs
60 output reg rx_valid;
61 output reg [9:0] rx_data;
62 output reg MISO;
63 //internal signals
64 reg add_or_data;
65 reg [3:0] counter;
66 reg [9:0] data;
67 reg [7:0] read_data;
68 (* fsm_encoding = "one_hot" *)
69 reg [2:0] cs,ns;
```

```

70 //next state logic
71 always @(*) begin
72     case (cs)
73     IDLE:
74         if(!SS_n) begin
75             ns=CHECK_COMMAND;
76         end
77     else begin
78         ns=IDLE;
79     end
80     CHECK_COMMAND: begin
81         if (!SS_n) begin
82             if (!MOSI) begin
83                 ns=WRITE;
84             end
85             else begin
86                 if (!add_or_data) begin
87                     ns=READ_ADD;
88                 end
89                 else begin
90                     ns=READ_DATA;
91                 end
92             end
93         end
94     else begin
95         ns=IDLE;
96     end
97 end
98 WRITE:
99     if (SS_n) begin
100         ns=IDLE;
101     end
102     else begin
103         ns=WRITE;
104     end
105 READ_ADD:
106     if (SS_n) begin
107         ns=IDLE;
108     end
109     else begin
110         ns=READ_ADD;
111     end
112 READ_DATA:
113     if (SS_n) begin
114         ns=IDLE;
115     end
116     else begin
117         ns=READ_DATA;
118     end
119 endcase
120 end
121 //state memory

```

```

130 //output logic
131 always @(posedge clk) begin
132     if (!rst_n) begin
133         MISO<=0;
134         rx_valid<=0;
135         rx_data<=0;
136         counter<=0;
137         data<=0;
138         add_or_data<=0;
139         read_data<=0;
140     end
141     else begin
142         if (cs==IDLE) begin
143             rx_valid<=0;
144             counter<=0;
145         end
146         else if (cs==WRITE) begin
147             if (counter==10) begin
148                 counter<=0;
149                 rx_valid<=1;
150                 rx_data<=data;
151             end
152             else begin
153                 counter<=counter+1;
154                 data<={data[8:0],MOSI};
155             end
156         end
157         else if (cs==READ_ADD) begin
158             if (counter==10) begin
159                 counter<=0;
160                 rx_valid<=1;
161                 rx_data<=data;
162                 add_or_data<=1;
163             end
164             else begin
165                 counter<=counter+1;
166                 data<={data[8:0],MOSI};
167             end
168         end

```

```

169         else begin
170             if (tx_valid) begin
171                 if (counter==0) begin
172                     counter<=counter+1;
173                 end
174                 if (counter==1) begin
175                     MISO<=tx_data[7];
176                     read_data<=tx_data<<1;
177                     counter<=counter+1;
178                 end
179                 else begin
180                     MISO<=read_data[7];
181                     read_data<=read_data<<1;
182                 end
183             end
184             else begin
185                 if (counter==10) begin
186                     counter<=0;
187                     rx_valid<=1;
188                     rx_data<=data;
189                 end
190                 else begin
191                     counter<=counter+1;
192                     data<={data[8:0],MOSI};
193                 end
194             end
195         end
196     end
197 end
198 endmodule

```

SPI Wrapper:

```
200 module SPI_WRAPPER (clk,rst_n,MOSI,SS_n,MISO);
201 //inputs
202 input clk;
203 input rst_n;
204 input MOSI;
205 input SS_n;
206 //outputs
207 output MISO;
208 //internal signal
209 wire [9:0] rx_data_to_din;
210 wire rx_valid;
211 wire [7:0] tx_data_to_dout;
212 wire tx_valid;
213 single_port_sync_ram ram (rx_data_to_din,rx_valid,clk,rst_n,tx_valid,tx_data_to_dout);
214 SPI_slave spi (MOSI,SS_n,tx_data_to_dout,tx_valid,clk,rst_n,MISO,rx_data_to_din,rx_valid);
215 endmodule
```

Test Bench:

```
1  module spi_wrapper_tb();
2  reg clk;
3  reg rst_n;
4  reg MOSI;
5  reg SS_n;
6  wire MISO;
7  reg [7:0] address;
8  reg [7:0] data;
9  reg [7:0] dummy_data;
10 // Instantiate DUT
11 SPI_WRAPPER DUT (clk,rst_n,MOSI,SS_n,MISO);
12 // Clock generation
13 initial begin
14     clk = 0;
15     forever #1 clk = ~clk;
16 end
17 // Test values
18 initial begin
19     address      = 8'd106;
20     data          = 8'hA5;
21     dummy_data   = 8'hF1;
22 end
23 integer j;
24 initial begin
25     // preload memory
26     $readmemh("mem.dat", DUT.ram.mem);
27
28     // reset
29     rst_n = 0;
30     MOSI  = 1;
31     SS_n  = 1;
32     repeat(3) @(negedge clk);
33     rst_n = 1;
34
35     // Write Address Test
36     SS_n = 0;
37     @(negedge clk);
38     MOSI = 0; @(negedge clk);
39     MOSI = 0; @(negedge clk);
```

```

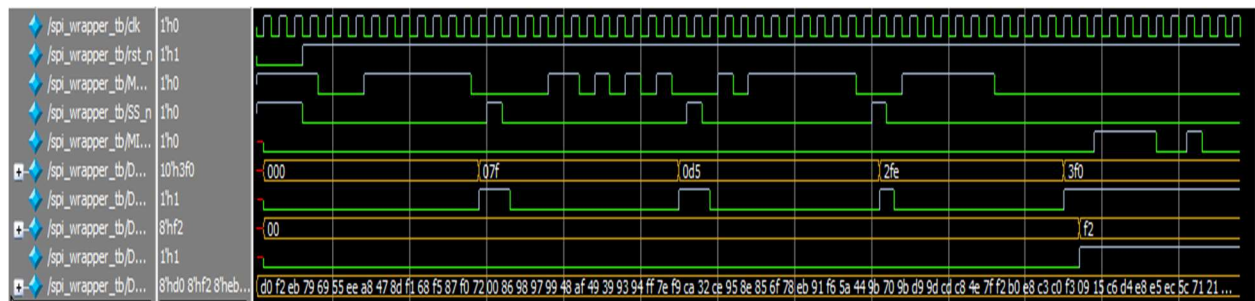
40     MOSI = 0; @(negedge clk);
41     for (j = 0; j < 8; j = j + 1) begin
42         MOSI = address[7-j];
43         @(negedge clk);
44     end
45     SS_n = 1; @(negedge clk);
46
47     // Write Data Test
48     SS_n = 0;
49     @(negedge clk);
50     MOSI = 0; @(negedge clk);
51     MOSI = 0; @(negedge clk);
52     MOSI = 1; @(negedge clk);
53     for (j = 0; j < 8; j = j + 1) begin
54         MOSI = data[7-j];
55         @(negedge clk);
56     end
57     SS_n = 1; @(negedge clk);
58
59     // Read Address Test
60     SS_n = 0;
61     @(negedge clk);
62     MOSI = 1; @(negedge clk);
63     MOSI = 0; @(negedge clk);
64     for (j = 0; j < 8; j = j + 1) begin
65         MOSI = address[7-j];
66         @(negedge clk);
67     end
68     SS_n = 1; @(negedge clk);
69
70     //Read Data Test
71     SS_n = 0;
72     @(negedge clk);
73     MOSI = 1; @(negedge clk);
74     MOSI = 1; @(negedge clk);
75     for (j = 0; j < 8; j = j + 1) begin
76         MOSI = dummy_data[7-j];
77         @(negedge clk);
78     end
79
80     // wait for data on MISO
81     repeat(12) @(negedge clk);
82
83     $stop;
84 end
85 endmodule

```

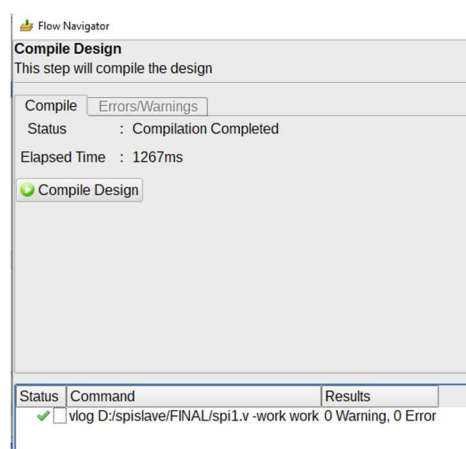
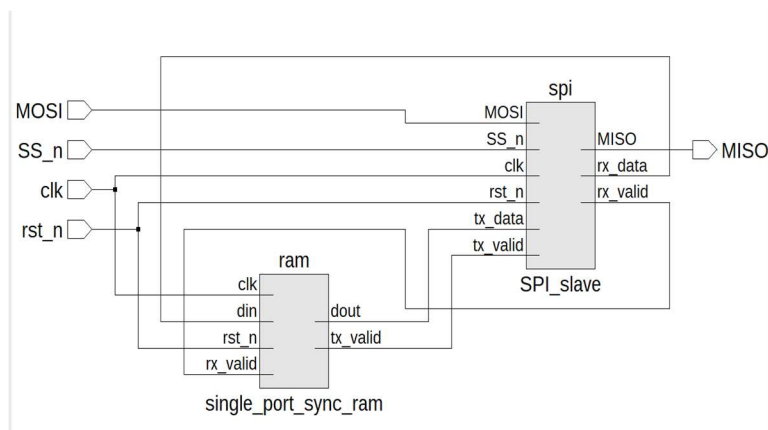
DO File:

```
1 vlib work
2 vlog spi1.v sp1tb.v
3 vsim -voptargs=+acc spi_wrapper_tb
4 add wave sim:/spi_wrapper_tb/clk
5 add wave sim:/spi_wrapper_tb/rst_n
6 add wave sim:/spi_wrapper_tb/MOSI
7 add wave sim:/spi_wrapper_tb/SS_n
8 add wave sim:/spi_wrapper_tb/MISO
9 add wave sim:/spi_wrapper_tb/DUT/rx_data_to_din
10 add wave sim:/spi_wrapper_tb/DUT/rx_valid
11 add wave sim:/spi_wrapper_tb/DUT/tx_data_to_dout
12 add wave sim:/spi_wrapper_tb/DUT/tx_valid
13 add wave sim:/spi_wrapper_tb/DUT/ram/mem
14 run -all
```

Questa Sim Waveform:



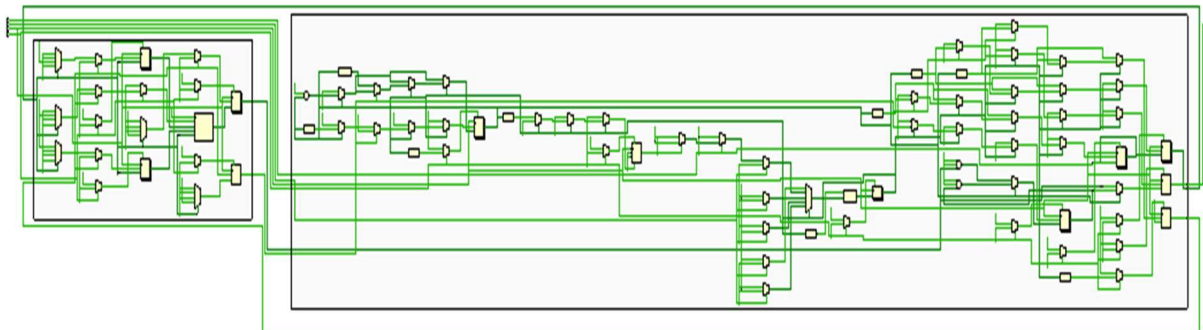
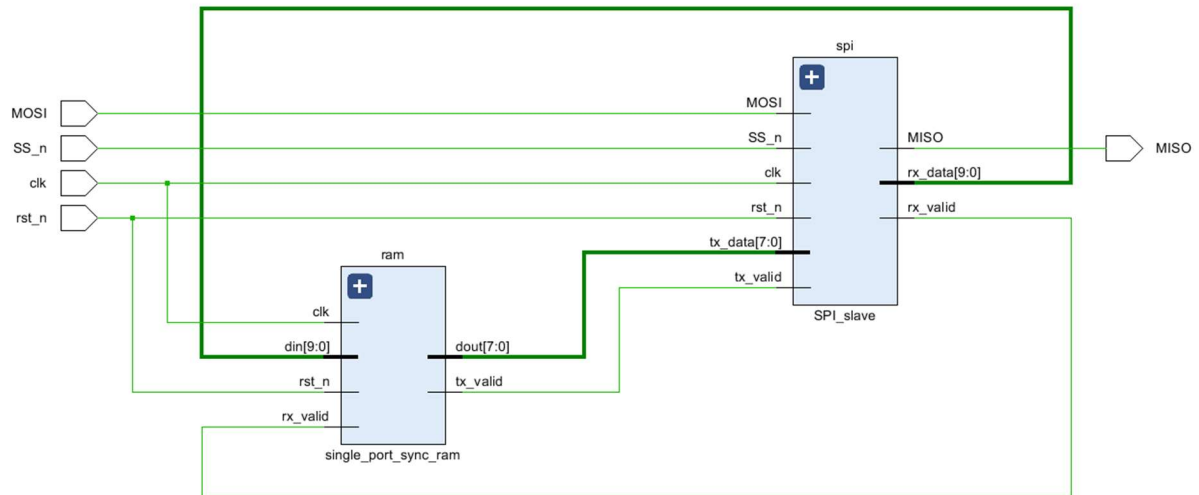
Linting:



Constraint File:

```
1  ## This file is a general .xdc for the Basys3 rev B board
2  ## To use it in a project:
3  ## - uncomment the lines corresponding to used pins
4  ## - rename the used ports (in each line, after get_ports) according to the top level signal names in the pro
5
6  ## Clock signal
7  set_property -dict {PACKAGE_PIN W5 IOSTANDARD LVCMOS33} [get_ports clk]
8  create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add [get_ports clk]
9
10
11  ## Switches
12  set_property -dict {PACKAGE_PIN V17 IOSTANDARD LVCMOS33} [get_ports rst_n]
13  set_property -dict {PACKAGE_PIN V16 IOSTANDARD LVCMOS33} [get_ports SS_n]
14  set_property -dict {PACKAGE_PIN W16 IOSTANDARD LVCMOS33} [get_ports MOSI]
15  #set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports {sw[3]}]
16  #set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports {sw[4]}]
17  #set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports {sw[5]}]
18  #set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports {sw[6]}]
19  #set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports {sw[7]}]
20  #set_property -dict { PACKAGE_PIN V2 IOSTANDARD LVCMOS33 } [get_ports {sw[8]}]
21  #set_property -dict { PACKAGE_PIN T3 IOSTANDARD LVCMOS33 } [get_ports {sw[9]}]
22  #set_property -dict { PACKAGE_PIN T2 IOSTANDARD LVCMOS33 } [get_ports {sw[10]}]
23  #set_property -dict { PACKAGE_PIN R3 IOSTANDARD LVCMOS33 } [get_ports {sw[11]}]
24  #set_property -dict { PACKAGE_PIN W2 IOSTANDARD LVCMOS33 } [get_ports {sw[12]}]
25  #set_property -dict { PACKAGE_PIN U1 IOSTANDARD LVCMOS33 } [get_ports {sw[13]}]
26  #set_property -dict { PACKAGE_PIN T1 IOSTANDARD LVCMOS33 } [get_ports {sw[14]}]
27  #set_property -dict { PACKAGE_PIN R2 IOSTANDARD LVCMOS33 } [get_ports {sw[15]}]
28
29
30  ## LEDs
31  set_property -dict {PACKAGE_PIN U16 IOSTANDARD LVCMOS33} [get_ports MISO]
```

Elaborated Design:



FSM Encoding:

One Hot:

➤ FSM Encoding Table :

State	New Encoding	Previous Encoding
IDLE	00001	000
CHECK_COMMAND	00010	001
WRITE	00100	010
READ_ADD	01000	011
READ_DATA	10000	100

➤ Timing Report after Synthesis:

Design Timing Summary

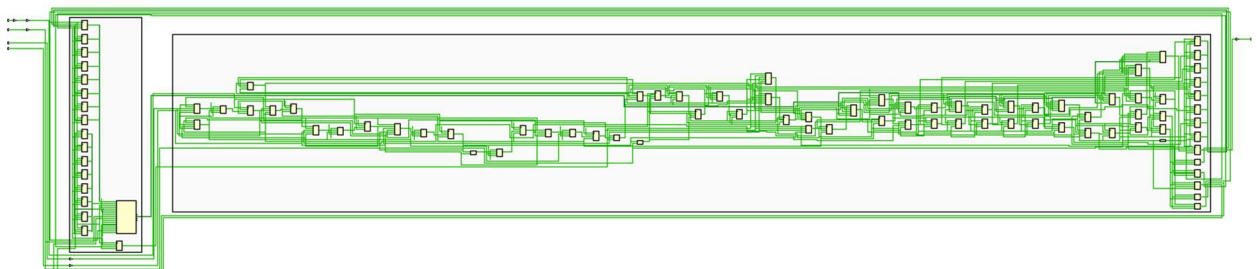
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.443 ns	Worst Hold Slack (WHS): 0.146 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 129	Total Number of Endpoints: 129	Total Number of Endpoints: 59

All user specified timing constraints are met.

➤ Utilization Report after Synthesis:

Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	30	61	0.5	5	1
ram (single_port_sync...	0	17	0.5	0	0
spi (SPI_slave)	30	44	0	0	0

➤ Schematic after Synthesis:



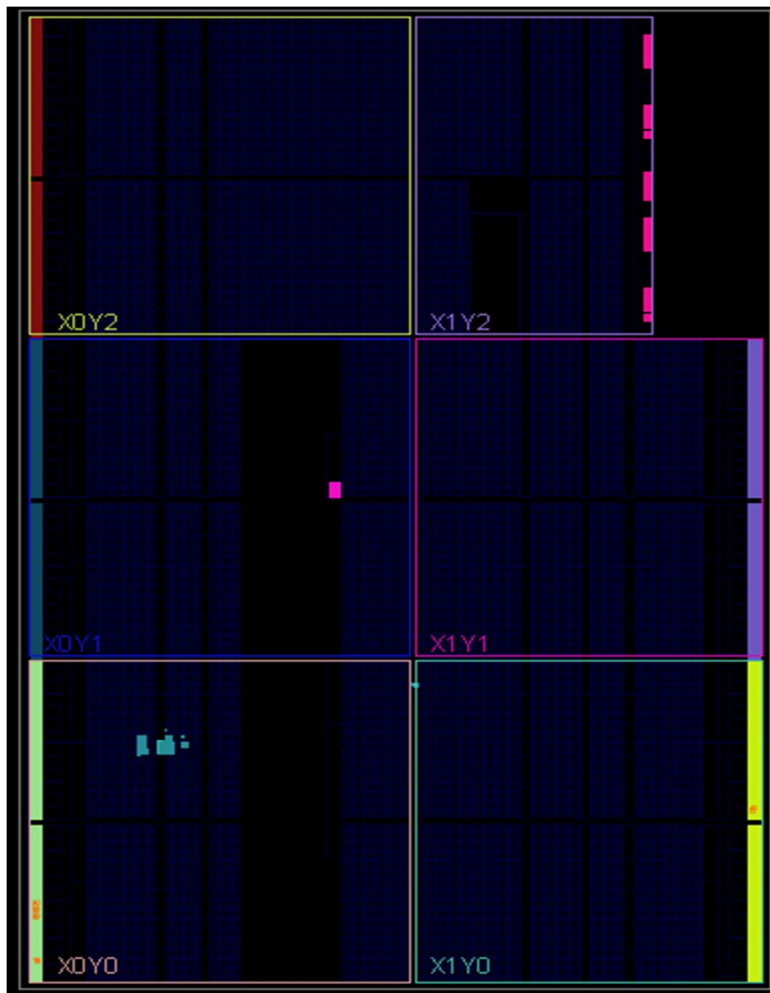
➤ Timing Report after Implementation:

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 5.686 ns		Worst Hold Slack (WHS): 0.042 ns		Worst Pulse Width Slack (WPWS): 4.500 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 130		Total Number of Endpoints: 130		Total Number of Endpoints: 59	
All user specified timing constraints are met.					

➤ Utilization Report after Implementation:

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	30	61	17	30	15	0.5	5	1
ram (single_port_sync...	1	17	3	1	0	0.5	0	0
spi (SPI_slave)	29	44	15	29	14	0	0	0

➤ Device:



Gray:

➤ FSM Encoding Table :

State	New Encoding	Previous Encoding
IDLE	000	000
CHECK_COMMAND	001	001
WRITE	011	010
READ_ADD	010	011
READ_DATA	111	100

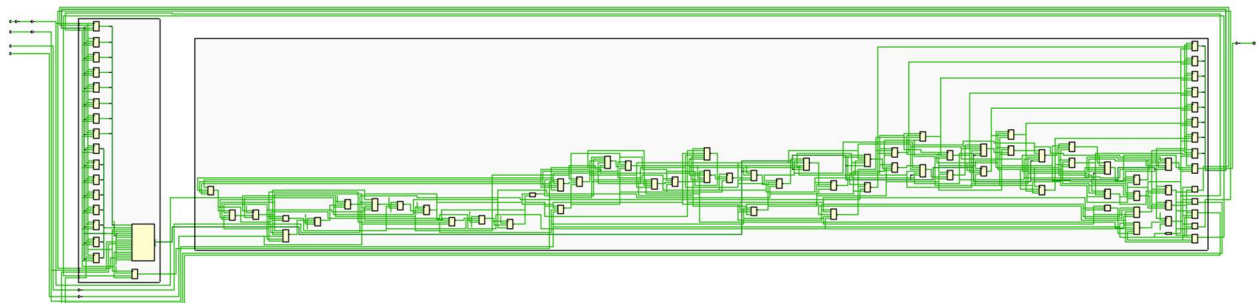
➤ Timing Report after Synthesis:

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 6.471 ns	Worst Hold Slack (WHS): 0.152 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 129	Total Number of Endpoints: 129	Total Number of Endpoints: 57
All user specified timing constraints are met.		

➤ Utilization Report after Synthesis:

Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	30	57	0.5	5	1
ram (single_port_sync...	0	17	0.5	0	0
spi (SPI_slave)	30	40	0	0	0

➤ Schematic after Synthesis:



➤ Timing Report after Implementation:

Design Timing Summary

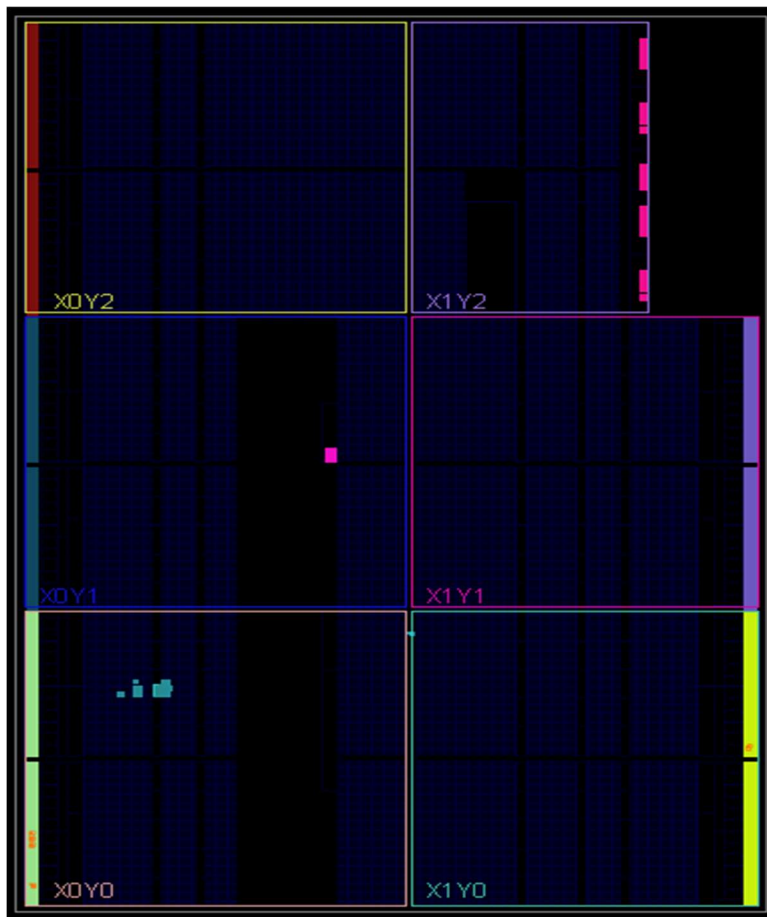
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 5.246 ns	Worst Hold Slack (WHS): 0.065 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 130	Total Number of Endpoints: 130	Total Number of Endpoints: 57

All user specified timing constraints are met.

➤ Utilization Report after Implementation:

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	31	57	13	31	17	0.5	5	1
ram (single_port_sync...	1	17	4	1	0	0.5	0	0
spi (SPI_slave)	30	40	12	30	16	0	0	0

➤ Device:



Sequential:

➤ FSM Encoding Table :

State	New Encoding	Previous Encoding
IDLE	000	000
CHECK_COMMAND	001	001
WRITE	010	010
READ_ADD	011	011
READ_DATA	100	100

➤ Timing Report after Synthesis:

Design Timing Summary

Setup

Worst Negative Slack (WNS): 6.471 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 129

Hold

Worst Hold Slack (WHS): 0.152 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 129

Pulse Width

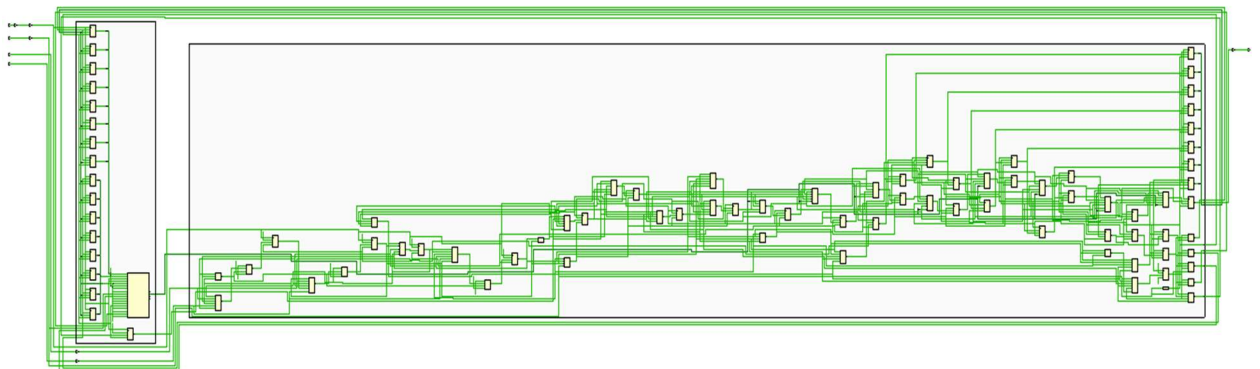
Worst Pulse Width Slack (WPWS): 4.500 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 57

All user specified timing constraints are met.

➤ Utilization Report after Synthesis:

Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	30	57	0.5	5	1
ram (single_port_sync...	0	17	0.5	0	0
spi (SPI_slave)	30	40	0	0	0

➤ Schematic after Synthesis:



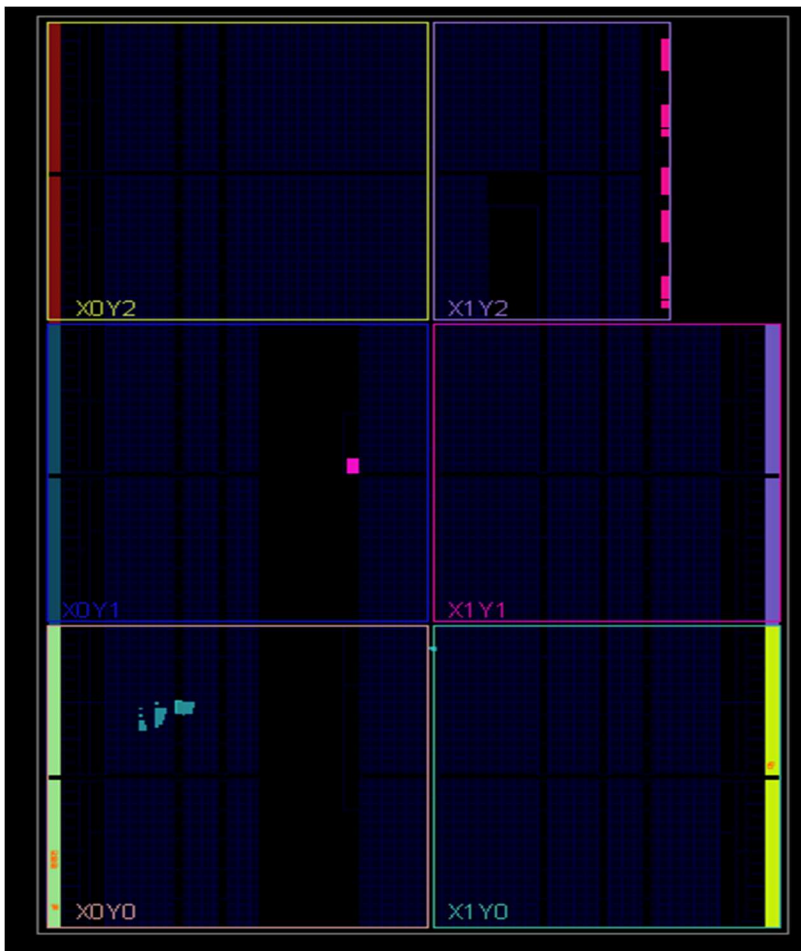
➤ Timing Report after Implementation:

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS):	5.108 ns	Worst Hold Slack (WHS):	0.089 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	130	Total Number of Endpoints:	130
			57
All user specified timing constraints are met.			

➤ Utilization Report after Implementation:

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N SPI_WRAPPER	31	57	18	31	19	0.5	5	1
ram (single_port_sync...	1	17	4	1	0	0.5	0	0
spi (SPI_slave)	30	40	16	30	18	0	0	0

➤ Device:



Since the design is an SPI controller, meeting timing is critical for reliable high-speed data transfer. After comparing FSM encodings, the one-hot scheme achieved the largest Worst Negative Slack (WNS), which indicates the best timing margin. Therefore, **one-hot** encoding will be chosen to ensure the fastest and most robust operation.